Axona

Building Applications That Have No Server

David A. Smith

Axona.net

The Axona Programmer Guide v4.48.0 2026-07-27

The Axona Programmer Guide

Build apps that have no server. *(kernel 4.48.0 · testnet)*

You're about to build something unusual: an application with no backend. No database to stand up, no message broker to rent, no API keys, nothing to deploy before your first user shows up. Your users' browsers and devices *are* the infrastructure — they find each other, carry each other's messages, and keep the whole thing alive.

This guide is for application programmers. You do not need to know how distributed hash tables work, and this guide will not make you learn. (If the machinery genuinely interests you, it's all in Under the hood at the end — strictly optional.)

Companions:

- Quick Start — a working roundtrip in 5 minutes. Do it first.
- API Reference — every signature, exactly.
- AI Grounding — building with an AI assistant? Hand it this file.
- Services Guide — running bridges and relays (you can skip this for a long time).

Building with an AI? Most Axona apps are. Paste Axona-AI-Grounding-v4.48.0.md into your assistant's context and it will know every rule in this guide. Keep this guide for *you* — it explains the why.

Table of contents

1. The five ideas
2. First contact
3. Idea by idea, gently
4. Recipes
5. Things that will bite you
6. Errors, when they happen
7. Limits worth knowing
8. Going live
9. Under the hood *(optional)*
10. Appendix: what's coming, and migrating from older lines

---

1. The five ideas

Everything you'll ever do with Axona is a combination of five ideas. Here they are in full; the rest of this guide is elaboration.

1. **A peer.** Your app running, connected to the network. It has a *connection identity* minted fresh each run — think of it as the seat your app happens to be sitting in today.

2. **An author.** A signing key that says *who wrote this.* It has no location, works from any device, and is the only thing worth saving to disk. If your app has "users," an author key is a user.

3. **A topic.** A place messages go. Not a string — a small object: `{ region: 'useast', name: 'lobby' }`. Anyone who writes the same fields gets the same place. No registration, no server assigning channels: shared fields *are* the rendezvous.

4. **Publish.** Send a message to a topic, signed by an author: `peer.pub(topic, message, { signWith: author })`.

5. **Subscribe.** Receive a topic's messages, live and (if you ask) recent history: `peer.sub(topic, handler, { since: 'all' })`.

That's the protocol. Chat, feeds, presence, file sharing, multiplayer state, agent-to-agent traffic — they're all arrangements of these five.

2. First contact

If you haven't run the Quick Start, do it now — five minutes, a real message through the real network. The heart of it:

```
// after connect() (Quick Start step 4; one call — see §3.1)

const TOPIC = { region: 'useast', name: 'quick-start-demo' };

await peer.sub(TOPIC, (env) => {
  console.log('[recv]', env.message);
}, { since: 'all' });

await peer.pub(TOPIC, 'hello, everyone', { signWith: author });
```

You publish; the network routes it; everyone subscribed — including you — receives it. That echo of your own message is not an accident, and you'll learn to love it (§5, "There is no ack").

3. Idea by idea, gently

3.1 The peer — your seat at the table

A peer is created in one call, pointed at a *bridge* — the meeting point where peers first find each other (after that, they talk directly):

```
import { connect } from '@axona/protocol/connect.js';

const { peer, author, status, disconnect } = await connect({
  bridge:   'wss://testnet.axona.net',
  location: { lat: 38.0, lng: -77.0 },   // the user's real location
  author:   'myapp:author',              // persist the author key under this name
});
// status: { ready: true, peers: 13, ms: 452, reason: 'minPeers' }
```

That's it — identities minted, transport connected, mesh warmed. When your app closes, `await disconnect()` leaves gracefully.

Under the sugar — the primitives, if you need custom wiring

`connect()` composes public kernel pieces you can always use directly (a custom transport, several peers on one page, a persistence adapter):

```
import {
  AxonaPeer, AxonaDomain, NeuronNode,
  createNodeIdentity, createAuthorIdentity,
} from '@axona/protocol';
import { webTransport } from '@axona/protocol/transport/web/index.js';

const nodeIdentity = await createNodeIdentity({ lat: 38.0, lng: -77.0 });
const author       = await createAuthorIdentity({ persistAs: 'myapp:author' });
const transport    = webTransport({ bridgeUrl: 'wss://testnet.axona.net',
                                    identity: nodeIdentity });
const node = new NeuronNode({ id: nodeIdentity.id, lat: 38.0, lng: -77.0 });
node.transport = transport;
const peer = new AxonaPeer({ domain: new AxonaDomain({ k: 20 }),
                             node, nodeIdentity, transport });
await transport.start(nodeIdentity.id);
await peer.start();
await peer.ready();
```

Why the location? Axona is geographic: the first byte of every address is a region of the Earth, and traffic for a region's topics stays among that region's peers. Your user in Virginia is a `useast` peer; their local topics never detour through Frankfurt.

The connection identity is deliberately disposable — fresh each run, like a dynamic IP. Don't save it, don't display it, don't build on it. The thing with a lifespan is:

3.2 The author — the only key you keep

```
const author = await createAuthorIdentity({ persistAs: 'myapp:author' });
```

An author is an Ed25519 keypair. Its public half — `author.authorId` — is what other people recognize: it appears (as `signerPubkey`) on every message it signs, from any device, from any location, forever. Persist it (`persistAs` uses localStorage in the browser) and your user has a stable identity across sessions. Skip `persistAs` and you have a throwaway persona — also sometimes exactly what you want. One app can hold several authors at once (work self, home self, bot self).

There's no account creation, no email verification, no password reset — because there's no one to reset it *with*. The key is the identity. That's liberating and unforgiving in equal measure: lose the key, lose the identity. Persist it.

3.3 The topic — an address you compute, not a channel you create

```
const lobby = { region: 'useast', name: 'lobby' };                        // anyone writes
const feed  = { region: 'useast', owner: me.authorId, name: 'posts' };   // only I write
const inbox = { region: 'useast', owner: me.authorId, name: 'inbox', write: 'open' }; //
 ↪  anyone writes TO me
```

Those three shapes cover a startling share of application design:

- **The open room** (`{ region, name }`) — a chat, a game lobby, a public firehose. Anyone who knows the fields can read and write.
- **The owned feed** (`owner` present; write defaults to `'owner'`) — a blog, a profile, a bot's announcements. Everyone can read; only the owner's key can write, and the *network itself* enforces that — a forged post is refused by the peers storing the topic, not just by polite clients.
- **The inbox** (`owner` + `write: 'open'`) — messages *to* someone. Anyone writes; the owner's app reads.

Two rules keep topic bugs out of your life. First, **publisher and subscriber must use identical fields** — the topic ID is a hash of them, so `{ region: 'useast', name: 'lobby' }` and `{ name: 'lobby' }` are different places (the second defaulted its region to wherever the publisher happened to be). Centralize your descriptors in one function and call it from both sides. Second, **prefer naming the region explicitly** for anything two differently-located users must both find.

Need a shareable read handle? `await deriveTopicId(topic)` returns the 66-hex ID; `sub` and `pull` accept it directly.

3.4 Publish — signed, fire-and-forget, and that's a feature

```
const msgId = await peer.pub(topic, { text: 'shipped it' }, { signWith: author });
```

The message is any JSON-serializable value. The signature travels with it; every receiver verifies it before your handler ever sees it. The returned `msgId` is a *content address* — a hash of author + payload — which buys you two elegant behaviors free of charge: publishing the identical thing twice doesn't duplicate (it refreshes), and any message can later be fetched or retracted by its id.

To publish anonymously, say so out loud: `{ signWith: ANONYMOUS }`. Forgetting `signWith` entirely is an error — Axona never guesses who's speaking.

And no, there is no delivery callback. See §5 for why that's a feature and what to do instead (spoiler: you're already subscribed — watch your own message arrive).

3.5 Subscribe — live tail, with as much history as you ask for

```
const sub = await peer.sub(topic, (env) => {
  if (env.deleted) return removeFromUI(env.msgId);      // a retraction marker
  render(env.message, env.signerPubkey);                // verified author (or undefined)
}, { since: 'all' });
```

The `since` option is your replay dial:

`since`	You receive
(omitted)	live messages only, from now
`'latest'`	the newest cached message, then live
`'all'`	recent history (the cached queue, ~24 h), then live
`1719954000000`	everything newer than that timestamp, then live

Deliveries are exactly-once per message per subscriber — the kernel dedups by `msgId`, so you never write defensive "have I seen this?" code. `sub.stop()` ends one subscription; `peer.unsub(topic)` ends all of yours for that topic.

4. Recipes

Real shapes, ready to lift. Each assumes the assembled `peer` and `author` from §3.1.

4.1 A chat room

```
const room = (name) => ({ region: 'useast', name: `chat:${name}` });

async function join(name, onMessage) {
  return peer.sub(room(name), (env) => {
    if (env.deleted) return;
    onMessage({ text: env.message.text,
                from: (env.signerPubkey ?? 'anon').slice(0, 8),
                at:   env.ts });
  }, { since: 'all' });          // newcomers see recent history
}

async function say(name, text) {
  return peer.pub(room(name), { text }, { signWith: author });
}
```

That's a chat app. History for late joiners, live tail for everyone, authorship on every line — and notice what's missing: no room creation step. The first person to publish *is* the room springing into existence.

4.2 A personal feed (and reading someone else's)

```
const myFeed = { region: 'useast', owner: author.authorId, name: 'posts' };

await peer.pub(myFeed, { title: 'First post', body: '…' }, { signWith: author });
```

Only your key can write it — enforced by the network, not by convention. For a friend to read your feed they need two facts: your `authorId` and the region you keep it in. Publish those anywhere (a QR code, an inbox message, a chat), then:

```
const theirFeed = { region: 'useast', owner: theirAuthorId, name: 'posts' };
await peer.sub(theirFeed, (env) => renderPost(env.message), { since: 'all' });
```

4.3 An inbox

```
const inboxOf = (authorId) => ({ region: 'useast', owner: authorId, name: 'inbox', write:
 ↪  'open' });

// anyone sends:
await peer.pub(inboxOf(theirAuthorId), { from: author.authorId, text: 'hi!' },
               { signWith: author });
// the owner reads:
await peer.sub(inboxOf(author.authorId), (env) => showDM(env.message), { since: 'all' });
```

The envelope's `signerPubkey` tells the owner who *really* sent each message (the `from` field inside the payload is just a courtesy copy — trust the signature, display the field).

4.4 Who's here? (presence)

```
const presence = { region: 'useast', name: 'myapp:presence' };
const here = new Map();   // authorId -> last heartbeat

setInterval(() => peer.pub(presence, { at: Date.now() }, { signWith: author }), 30_000);

await peer.sub(presence, (env) => {
  if (env.signerPubkey) here.set(env.signerPubkey, env.ts);
});
// online = heartbeat within the last ~90s
const online = [...here].filter(([, t]) => Date.now() - t < 90_000);
```

Presence is just a topic everyone heartbeats into. Messages age out of the cache on their own; nobody cleans up.

Two details of this recipe are deliberate, and both have been gotten wrong in a shipped app: the subscription is **live-only** (no `since` — replaying hours of stale heartbeats is noise), and recency reads **`env.ts`** — the *publish* time — never the local clock at delivery. If you do replay history on a presence-like topic, a heartbeat that *arrives* now may have been *published* hours ago; stamping arrival time resurrects everyone who was online in the last day. Keep the latest `env.ts` per author and compare against that.

4.5 Taking it back (kill)

```
const msgId = await peer.pub(room('general'), { text: 'typo galore' }, { signWith: author });
await peer.kill(room('general'), msgId, { signWith: author });   // same author key!
```

Every subscriber's handler receives `{ deleted: true, msgId }` — drop it from your UI. Honesty clause: retraction is cooperative, not magical. A device that already displayed the message has, well, displayed it. And an anonymous message can never be killed — no key, no proof it's yours.

4.6 Sharing files (std/chunk)

Single publishes over ~15 KB are unreliable on real browser channels, so don't send big payloads — chunk them. The kernel ships the helper:

```
import { publishChunkedBytes, receiveChunkedBytes } from '@axona/protocol/std/chunk.js';

const drop = { region: 'useast', name: 'myapp:drops' };

await publishChunkedBytes(peer, fileBytes /* Uint8Array */, {
  topic: drop, signWith: author,
  meta: { filename: 'demo.png', mime: 'image/png' },
});

await receiveChunkedBytes(peer, drop, {
  onComplete: ({ bytes, meta }) => showImage(bytes, meta),
  onProgress: ({ received, total }) => bar.set(received / total),
});
```

4.7 Direct messages (device to device)

Pub/sub addresses *audiences*. To address one specific connected device:

```
peer.onMessage(async (fromNodeId, msg) => ({ pong: msg.ping }));   // respond
const reply = await peer.send(someNodeId, { ping: 1 });            // RPC
peer.notify(someNodeId, { fyi: true });                            // fire-and-forget
```

Node IDs come from `peer.peers()` / `peer.onPeerJoin`. Remember: a node ID is a *session*, not a person — for person-to-person, use an inbox (§4.3).

4.8 How busy is this topic?

```
import { metricTopic, deriveTopicId } from '@axona/protocol';

const snap = await peer.metrics(topic);        // one-shot { subscribers, current_count, ...
 ↪ }

await peer.sub(metricTopic(await deriveTopicId(topic)), (env) => {
  const m = JSON.parse(env.message);
  badge.textContent = `${m.subscribers} watching`;
}, { since: 'latest' });                       // live-updating counter
```

Metrics are **published on demand**: your subscription is what switches them on, and the root answers **immediately** — the first snapshot arrives at routing latency (~0.3 s on testnet), with or before your data replay, whether or not anyone has ever published (then one every ~20 s while you stay subscribed). Two habits worth keeping:

- **Treat silence as *unknown*, not zero.** Until the first snapshot arrives, "no activity" isn't an answer yet — a `current_count: 0` snapshot is.
- **Keep the subscription open.** A sub torn down seconds after it starts can still miss the answer under churn; the stream is the primitive, not a one-shot read.

Counts are advisory — decorate with them, never authorize with them.

4.9 Remembering your users

The complete persistence story for most apps is one line you've already written: `createAuthorIdentity({ persistAs: 'myapp:author' })`. Node identity? Re-mint each run. Subscriptions? Re-subscribe on startup (with `since`, you'll catch up on what you missed). Messages? The network holds the recent window; anything your app must keep beyond ~24 h, store like any other app data.

4.10 Playing nicely with other apps

Topics are shared space — another app can subscribe to your topic. If you want your messages legible to it (and its to you), use the standard body:

```
import { makeMessage, readMessage } from '@axona/protocol/std/message.js';

await peer.pub(topic, makeMessage('hello', { mood: 'sunny' }), { signWith: author });
const { text } = readMessage(env.message);    // tolerant of other apps' shapes
```

4.11 If your author is a bot, say so

```
await peer.setAuthorClass('agent', { signWith: author, label: 'ticker-bot' });
```

A voluntary, signed declaration that this key is operated by software. Readers resolve it with `getAuthorClass(authorId)` and can badge, rank, or filter. It gates nothing — it's honesty infrastructure, and well-behaved bots (and AI-built apps!) should use it.

4.12 Surviving sleep (session recovery)

What happens to your session when the laptop lid closes? The socket dies, the mesh heals around your absence, and the seat your subscriptions held at each root expires. The kernel reconnects the socket for you — with backoff from 1 s to a 16 s cap, re-running the handshake on every reopen — but that is one layer of several, and a session recovers only if *all* of them do. On 2026-08-05 a chat session slept overnight and woke showing "Seeking Peers" for ten hours. It received nothing. A message typed into it did something worse than fail: `pub()` resolved, the zero-peer node rooted the topic on itself, delivered its own echo, and the message rendered as sent — while existing nowhere but that one machine's memory.

Your app is the only layer that knows what "healthy" means for it, so your app owns the last line of recovery. The pattern has four parts.

Detect. Two signals, both cheap. A periodic timer whose tick arrives far later than scheduled means the machine slept — timers don't drift 30 s on a live page. And `peer.peers().length` sitting

at zero after the session was once connected means it's wedged, whatever the cause below. Check immediately on `visibilitychange` and `online` too; that's the moment the user is looking.

```
let last = Date.now(), zeroSince = null, wasConnected = false;
setInterval(async () => {
  const gap = Date.now() - last; last = Date.now();
  const peers = peer.peers().length;
  if (peers > 0) { wasConnected = true; zeroSince = null; return; }
  if (!wasConnected) return;                    // still bootstrapping
  zeroSince ??= Date.now();
  const slept = gap > 30_000;
  const stuck = navigator.onLine !== false && Date.now() - zeroSince > 25_000;
  if (slept || stuck) await recoverSession();   // see below
}, 5_000);
```

Recover by rebuilding, not by nudging. The wedge has several distinct causes — the reconnect loop died, the socket came back but the mesh never rebuilt, a graduated client whose re-dial watchdog failed — and your app cannot tell them apart from outside. A full teardown and reconnect recovers all of them: `disconnect()`, `connect()` again, re-subscribe every topic (with `since`, you catch up on what you missed while deaf). Back off between attempts so a genuinely-down network doesn't hot-loop.

Confirm sends honestly. Track each publish as *pending* until your own `msgId` returns through your subscription **while `peers() > 0`**. The island echo arrives even on a dead session and must not count. Show pending state in your UI — a message that never confirmed should not look like one that did.

Replay after recovery. Here the protocol pays you back: `msgId = hash(publisher + message)`, and `makeMessage` bodies carry no timestamp — so re-publishing the *same payload object* under the *same author* produces the *same msgId*. Retry is idempotent by construction. A stranded message replays safely; one that actually landed dedups at the root and simply confirms. Keep the exact payload with each pending record and re-`pub()` them all once the rebuilt session is confirmed healthy. (If you build payloads with your own timestamps inside, you lose this — another reason to let the root do the stamping.)

Every long-lived Axona session needs this — browser tab, phone, laptop, or a bot on a server that suspends. The reference implementation is axona-chat v0.47.0 (`PeerContext.jsx` for the watchdog, `AxonaChatClient.js` for pending sends and replay).

5. Things that will bite you

Learn these here rather than in production. The first five are the classic protocol traps; the last five have each already bitten a real, shipped Axona application.

1. Two descriptors, one character apart, are two different topics. No error, no warning — just silence. The classic: publisher says `{ region: 'useast', name: 'lobby' }`, subscriber says `{ name: 'lobby' }` from a laptop in Berlin, and its defaulted region is `eucentral`. One shared

`topics.js` with functions like `room(name)` ends this bug forever.

2. There is no ack — stop waiting for one. `pub()` resolves when the message is *sent*, not delivered, and no receipt ever comes (it would stitch your connection to your authorship — an identity leak Axona refuses on principle). The idiom: you're subscribed to what you publish; when your own `msgId` comes back through your handler *and your peer has mesh peers*, it went through. That second condition is load-bearing: a peer with zero mesh peers roots the topic on itself, and the echo it delivers proves only that your own process heard you. See the session-recovery recipe for the full treatment. The kernel meanwhile retries quietly on your behalf.

3. Publishing before `ready()` is a coin flip. The mesh takes a few seconds to warm after `start()`. `await peer.ready()` — that's what it's for. (Your first publish would *probably* survive anyway; the kernel bursts a newcomer's first messages. "Probably" is not a UX.)

4. Messages are not rows. The network keeps a rolling ~24-hour window per topic, and open topics quota each author's share of it. Build for a living stream: late joiners see recent history, not the archive. Need an archive? That's your app's job (and re-publishing refreshes a message you want kept warm).

5. 15 KB. Above it, real-world WebRTC channels start dropping frames even though the hard cap is 256 KB. `std/chunk` exists so you never think about this again.

6. Delivery time is not publish time. A subscription with `since` replays history: a message that arrives *now* may have been published hours ago. Any recency logic — "last seen", online lists, freshness sorting — reads `env.ts` (the publish time), never `Date.now()` at delivery. The observed failure: an app stamped arrival time on replayed heartbeats, and every user from the past day appeared "online" for 90 seconds after each launch.

7. Switching users is not reconnecting. One peer serves any number of authors — authorship is per-call (`{ signWith }`), and swapping the active persona should touch *nothing* about the connection. The observed failure: an app tore down and rebuilt its peer on every persona switch, churning the mesh and killing live connections mid-negotiation for zero benefit.

8. Sharing a topic means sharing the whole descriptor. An invite, a directory advertisement, a QR code, a pasted link — whatever carries a topic to another user must carry `{ region, owner, name, write }` *complete*. The observed failure: a discovery ticker advertised owned topics by name and region only; joiners derived a different (ownerless) topic ID and landed in a working-looking, permanently empty room.

9. When derivation fails, fail. If the kernel rejects a descriptor, show the error and skip the topic. The observed failure: an app caught the throw and substituted a locally invented ID "so the UI kept working" — which converted a visible error into invisible, permanent non-delivery that no log would ever explain.

10. Local dev: bind the dev server to IPv4. Vite's default `localhost` binding lands on IPv6 `::1`, and Firefox gathers **zero** ICE candidates on a page served from a `::1` origin — every mesh dial fails with a misleading "your TURN server appears to be broken" console error, while Chromium works. One line ends it: `server: { host: '127.0.0.1' }`. (Bundler note: alias `node-datachannel` and

`node-datachannel/polyfill` to an empty stub for browser builds, and clear the bundler's dependency cache after changing the kernel pin.)

6. Errors, when they happen

Everything the kernel throws is an `AxonaError` with a stable `.code` — switch on codes, not message text:

Code	You did	Do instead
`PUBLISH_NO_PUBLISH_IDENTITY`	`pub` without `signWith`	pass an author, or `ANONYMOUS` explicitly
`WRITE_POLICY_VIOLATION`	wrote to someone else's owned topic	only the owner key writes; use their inbox
`TOPIC_REGION_REQUIRED`	derived an open topic with no region anywhere	name a `region`
`PUBLISH_PAYLOAD_TOO_LARGE`	>256 KB publish	`std/chunk`
`KILL_SIGN_FAILED`	killed with a different key	retract with the key that signed
WS close `4426`	version mismatch with the bridge	reinstall the pinned kernel tag

Not errors, by design: `pull()` returning `null` (gone or never existed) and `kill()` resolving `{ ok: false }` (nothing to retract).

7. Limits worth knowing

Thing	Value
Reliable message size	**15 KB** (chunk above; 256 KB absolute)
Message lifetime	~24 h (48 h max; re-publish to refresh)
History for late joiners	the cached queue — recent, bounded, not forever
One author's share of an open topic	quota-bounded (no flooding)
Clock skew tolerance on live publishes	±5 minutes (keep device clocks sane)

8. Going live

- **Point at the right network.** Both live networks run this guide's 4.x line: **testnet** (`wss://testnet.axona.net`) tracks the newest kernel, **production** (`wss://bridge.axona.net`, with a federated west sibling) runs the most recently promoted release, typically one behind. Develop against testnet; deploy against production. A genuinely mismatched kernel is refused loudly (close 4426) — match your kernel pin to your bridge.
- **HTTPS is mandatory** in the browser (`crypto.subtle`).

- **Prove it with two clients before you call it done.** A distributed app can pass every single-client test while being completely wrong — your own client shows you your own state, not the network's. The minimum gate: two clients in one topic exchanging messages; a kill on one disappearing on the other; a topic shared through your app's own invite/ad/QR surface opening with *content* (not a clean-looking empty room — that's a partial descriptor); a fresh client's presence view showing only the currently active; and the whole gate run in both a Chromium browser and Firefox. Every field failure in §5's entries 6–10 was invisible to one client.
- **Surface your versions.** Show the kernel version somewhere findable (`KERNEL_VERSION` import) — future-you debugging a mixed fleet will be grateful.
- **Durability beyond browser tabs.** An app whose topics matter when no user is online wants a small always-on peer that `host()`s them — that's a *relay*, a five-minute Node deployment. When you reach that point, the Services Guide is yours; not before.

9. Under the hood — optional

You can build everything above without this section. For the curious:

The mesh. Peers connect via a bridge (a meeting point, not a server — it relays introductions and can pass messages, but owns nothing), then directly to each other over WebRTC. Each peer maintains a small routing table; the network is navigable without anyone holding a map. It self-heals: dead connections are detected by heartbeat and routed around, no configuration, no ops.

The tree. Every topic ID is an address in the same space as peer IDs. The live peer *closest* to a topic's address automatically becomes its **root** — the coordinator that stamps, caches, and fans out that topic's messages. Nobody elects it; proximity appoints it. Popular topics grow a distribution tree (the root delegates batches of subscribers to child relays), so no node ever serves more than a bounded number directly.

Durability. The root continuously copies its cache to the peers next in line; when a root vanishes (a laptop closes), a warm backup notices within a minute and takes over, and every subscriber's periodic renewal re-attaches automatically. A peer that leaves *gracefully* hands every topic it was coordinating to an heir before it goes. And whenever a topic's coordinator changes hands, the old and new holders reconcile to the **union** of their history — so a newcomer asking for "everything" gets the whole timeline, not whichever half its node happened to hold. Your app sees, at worst, a brief pause. Your messages are also retried by your own peer until it observes them delivered. All of this is why the guide keeps saying *you don't manage the network* — it manages itself, aggressively.

Regions. The first byte of every ID is one of 192 real geographic cells, so a region's topics naturally live on that region's peers — locality without a CDN. (A stricter mode, where a topic may *only* be served in its own region, is built and waiting for network density to justify switching on.)

The full story — with diagrams — is the Axona Architecture note.

Nodes can refuse work (kernel 4.46.0+)

A node is allowed to say **no** to a role. If it is newly joined (still in its grace period), or genuinely

at capacity, it declines and the role goes to someone who can serve it. Three things follow that matter when you are reading logs or reasoning about delivery:

- **Refusal is not loss.** A declined publish is forwarded to a node that can take it. Only if there is *no* such node does it stop — and then it is logged `undeliverable` rather than retried, so it shows up instead of hanging.
- **The floor overrides refusal when there is no alternative.** You will see `admitted-despite` in logs: the node took a role it did not want because dropping the data was the worse option. That is working as designed, not a bug.
- **Capacity is measured, not counted.** A node with hundreds of roles that services them on time is healthy. Judge nodes by `servicePressure` / `helloPressure` from `peer.health().admission.capacity`, never by role count.

One refusal is absolute: **a bridge never holds a topic role**, at any load. See the Services Guide.

10. Appendix

Migrating from the v2/v3 lines

Only relevant if you have code from early 2026. The v3 flag-day replaced `deriveIdentity` with the two factories, string topics with descriptors, and `publisher`/`sign:false` with per-publish `signWith`; v4 changed nothing in the app API — it rebuilt reliability underneath (single-root axon trees, cohort replication, cold-publish burst, nearest-replica reads). `unpub` was removed and `touch` retired (re-publish to refresh a message instead). If a symbol you remember is missing from the API Reference, it didn't survive v3 — the reference is the source of truth.
